Foundations of Mathematics: Set Theory, Category Theory, and Type Theory

N-20230710

§1.1 The question of foundations

Three major foundational languages—set theory, category theory, and type theory—answer the same basic questions in different ways. The aim is not to declare a single winner, but to compare their answers:

(i) What are mathematical objects?

(ii) What counts as equality?

(iii) What kind of reasoning is allowed?

(iv) How does computation enter mathematics?

(v) How does one organize structures and transformations between them?

Set theory emphasizes membership; category theory emphasizes morphisms and universal properties; type theory emphasizes construction, judgment, and computation. The most interesting modern developments, especially univalent foundations and homotopy type theory, no longer keep these worlds separate.

§1.2 Set theory

§1.2.i From naive comprehension to axioms

Modern set theory begins with the intuition that mathematical objects can be collected into sets. In naive set theory, one might try to form

$$\{x \mid P(x)\}$$

for any property P. Russell's paradox shows that unrestricted comprehension is impossible: the class

$$R = \{x \mid x \notin x\}$$

leads to contradiction if it is treated as a set.

The response is axiomatization. Zermelo–Fraenkel set theory with Choice, ZFC, restricts set formation and supplies a controlled universe in which ordinary mathematics can be developed.

§1.2.ii ZFC as the standard background

The familiar axioms include extensionality, pairing, union, power set, infinity, replacement, foundation, separation, and choice. Their roles may be summarized as follows:

Extensionality. A set is determined by its elements.

Pairing, union, power set. These build new sets from old ones.

Infinity. This guarantees an infinite set, usually giving the natural numbers.

Replacement. Definable images of sets are sets.

Foundation. Membership is well-founded and circular membership is excluded.

Choice. Products of nonempty families are nonempty; equivalently, every surjection has a right inverse.

> **Remark 1.2.1 —** The strength of ZFC is not that it matches everyday mathematical language perfectly. Its strength is that it gives a stable formal universe large enough for most mathematics and disciplined enough to avoid naive paradoxes.

§1.2.iii Applications and limitations

Set theory supports analysis, algebra, topology, measure theory, model theory, and essentially all ordinary mathematical constructions. It also enables transfinite methods: ordinals, cardinals, large cardinals, forcing, independence, and descriptive set theory.

But ZFC has philosophical and practical limitations. It represents everything as sets, even when this representation feels artificial. A group is not experienced as a nested set; it is experienced as a structure with operations. A topological space is not merely a set of ordered pairs defining open sets; it is a space with continuous maps. This motivates category theory.

§1.3 Category theory

§1.3.i From objects to morphisms

Category theory changes the focus from internal membership to external relations. A category consists of objects and morphisms, with associative composition and identities. In this language, mathematical structures are understood by the maps they admit.

Definition 1.3.1. A category $\mathcal{C}$ consists of objects, morphism sets $\mathcal{C}(X, Y)$, identity morphisms id_X, and composition

$$\mathcal{C}(Y, Z) \times \mathcal{C}(X, Y) \to \mathcal{C}(X, Z),$$

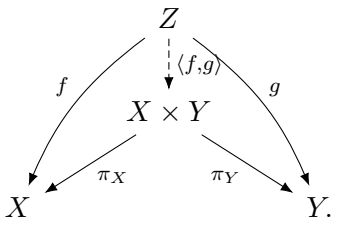
satisfying associativity and identity laws.

> **Example 1.3.2**
>
> **Set**, **Grp**, **Top**, **Ring**, and $\mathbf{Vect}_k$ are standard categories. Each has a different notion of structure-preserving map.

§1.3.ii Universal properties

The central categorical idea is the universal property. Products, coproducts, pullbacks, pushouts, quotients, tensor products, free objects, and completions can all be characterized by mapping properties.

For example, a product $X \times Y$ is characterized by



This diagram says that giving a map into a product is the same as giving compatible maps into its two factors.

> **Remark 1.3.3 —** Universal properties explain why the same construction appears in many different subjects. They reveal that the construction is not about the elements of a particular model, but about a structural role.

§1.3.iii Functors and natural transformations

Definition 1.3.4. A functor $F : \mathcal{C} \to \mathcal{D}$ sends objects to objects and morphisms to morphisms, preserving identities and composition.

Definition 1.3.5. A natural transformation $\eta : F \Rightarrow G$ between functors $F, G : \mathcal{C} \to \mathcal{D}$ assigns to each object X a morphism $\eta_X : F(X) \to G(X)$ such that for every $f : X \to Y$,

$$\begin{array}{ccc} F(X) & \xrightarrow{F(f)} & F(Y) \\ {\scriptstyle \eta_X}\downarrow & & \downarrow{\scriptstyle \eta_Y} \\ G(X) & \xrightarrow[G(f)]{} & G(Y) \end{array}$$

commutes.

This square is one of the simplest and most important categorical diagrams. It encodes the idea that a construction is canonical, not dependent on arbitrary choices.

§1.3.iv Categorical foundations

Category theory can function as a language for organizing mathematics, but it can also become foundational. ETCS axiomatizes sets categorically. Topos theory generalizes set-like universes. Higher category theory and infinity-categories extend the categorical language to homotopical and derived contexts.

> **Remark 1.3.6 —** Category theory does not merely replace set theory. It clarifies invariance. It asks not only what an object is made of, but how it transforms and how it participates in universal constructions.

§1.4 Type theory

§1.4.i Judgments and types

Type theory begins from judgments such as

$$a : A,$$

meaning that a is a term of type A. Instead of putting every object into one universe of sets, type theory stratifies objects by the types they inhabit.

> **Remark 1.4.1 —** The phrase "$a \in A$" in set theory and the judgment "$a : A$" in type theory look similar but behave differently. Membership is a proposition; typing is a judgment of well-formedness.

§1.4.ii Dependent types

Dependent type theory allows types to depend on terms. If A is a type and $B(x)$ is a type depending on $x : A$, then one can form:

$$\prod_{x:A} B(x), \qquad \sum_{x:A} B(x).$$

The product type corresponds to universal quantification, while the sum type corresponds to existential quantification.

§1.4.iii Curry–Howard

The Curry–Howard correspondence identifies propositions with types and proofs with terms:

$$\text{proposition} \leftrightarrow \text{type}, \qquad \text{proof} \leftrightarrow \text{term}.$$

For example,

$$A \times B$$

corresponds to conjunction, while

$$A \to B$$

corresponds to implication.

This makes type theory naturally computational. A proof is not merely a static certificate; it may carry an algorithm.

§1.4.iv Constructivity and computation

Classical set theory freely uses excluded middle and choice. Type theory is often constructive: to prove existence, one should construct a witness. This is not only a philosophical choice. It is what makes type theory suitable for proof assistants such as Coq, Lean, Agda, and Isabelle/HOL.

§1.5 Homotopy type theory and univalence

Homotopy type theory interprets types as spaces, terms as points, and identity proofs as paths. Higher identity proofs become higher homotopies. In this interpretation, equality is no longer merely a binary relation; it has structure.

The univalence axiom says, roughly, that equivalent structures may be identified:

$$(A = B) \simeq (A \simeq B).$$

This expresses a structuralist principle: isomorphic or equivalent objects should be interchangeable in mathematics.

Remark 1.5.1 — Univalence is one of the most striking attempts to synthesize set-theoretic rigor, categorical invariance, and type-theoretic computation. It turns the informal mathematical habit of identifying isomorphic objects into a formal principle.

§1.6 Comparative analysis

Framework	Primitive emphasis	Equality	Strength
Set theory	membership	extensional equality	universal coding power
Category theory	morphisms	isomorphism/ equivalence	structural invariance
Type theory	judgments/terms	identity types	computation and construction

Set theory is excellent as a common foundational background. Category theory is excellent for organizing structures and transformations. Type theory is excellent when proofs, programs, and constructions are meant to interact.

§1.7 Modern applications

§1.7.i Proof assistants

Proof assistants make foundations operational. Lean's mathlib, Coq's constructive type theory, Agda's dependent types, and Isabelle's higher-order logic show that foundational choices affect how mathematics is formalized.

§1.7.ii Computer science

Type theory underlies programming language theory, dependent types, formal verification, compiler correctness, and proof-carrying code. Category theory appears in semantics, monads, functional programming, and compositional systems. Set theory remains important in logic, model theory, semantics, and databases.

§1.7.iii Geometry and higher structures

Modern geometry often uses categorical and type-theoretic ideas: stacks, sheaves, derived categories, infinity-categories, and higher topos theory. Foundations are no longer merely a safety net beneath mathematics; they become active tools inside research.

§1.8 Two points expose three notions of sameness

Consider two sets

$$A = \{\text{red}, \text{blue}\}, \qquad B = \{0, 1\}.$$

Set-theoretically they are not equal: extensionality says that sets are equal precisely when their members coincide. Yet there are exactly two bijections $A \to B$:

	red	blue
f	0	1
g	1	0

The fact that the sets have the same cardinality does not tell us which labeling to use. That freedom is information, not an annoying technicality.

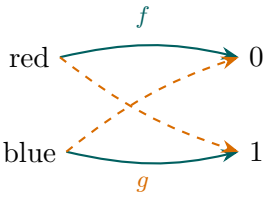


Figure 1.1: Two equally valid bijections. Literal equality, existence of an isomorphism, and choice of a particular isomorphism are different assertions.

Category theory keeps both bijections as distinct morphisms. If one set is viewed as a pointed set, with its first element designated, then morphisms must preserve that point; only one of the two bijections remains. What counts as an isomorphism depends on the structure one has asked the morphisms to respect.

§1.9 Why a product has no favorite encoding

In ZFC an ordered pair can be encoded by Kuratowski's formula

$$(a, b) = \{\{a\}, \{a, b\}\}.$$

Other encodings are possible. A categorical product instead has a role: for every pair of arrows $f : Z \to X$ and $g : Z \to Y$, there is exactly one arrow $\langle f, g\rangle : Z \to X \times Y$ whose projections are f and g.

The uniqueness clause has a concrete consequence. Suppose (P, p_X, p_Y) and (Q, q_X, q_Y) both satisfy the product property for X, Y. It supplies unique maps

$$u : P \to Q, \qquad v : Q \to P$$

such that

$$q_X u = p_X, \quad q_Y u = p_Y, \qquad p_X v = q_X, \quad p_Y v = q_Y.$$

Then vu and 1_P have the same composites with both projections of P. Since the defining property gives a *unique* map with these composites, $vu = 1_P$. Likewise $uv = 1_Q$. Thus the two products are isomorphic by a uniquely determined *projection-preserving* isomorphism.[1]

Without the qualifier about projections, the underlying product object may have other automorphisms. A universal property is powerful precisely because it tells us which comparisons are canonical once the required structure is fixed. This proof never had to inspect the set-theoretic implementation of an ordered pair.

§1.10 A dependent type is an executable promise

Now use the two-element type

$$\mathsf{Bool} = \{\mathsf{false}, \mathsf{true}\}.$$

[1] For this exact proof, see *Five Lectures on Category Theory*, Proposition 1.5.

An inhabitant of the dependent product

$$\prod_{b:\mathsf{Bool}} \mathbb{N}$$

must supply a natural number for *every* Boolean input. Evaluating such a function at the two inputs yields the map

$$h \longmapsto (h(\mathsf{false}), h(\mathsf{true})).$$

Conversely, a pair (m, n) defines

$$h_{m,n}(b) = \begin{cases} m, & b = \mathsf{false}, \\ n, & b = \mathsf{true}. \end{cases}$$

The two constructions are inverse up to function extensionality, so

$$\boxed{\left(\prod_{b:\mathsf{Bool}} \mathbb{N}\right) \simeq \mathbb{N} \times \mathbb{N}.}$$

This is the computational content of a universal statement: a proof that supplies an answer for each input acts as a program.

By contrast,

$$\sum_{b:\mathsf{Bool}} \mathbb{N} \simeq \mathbb{N} \sqcup \mathbb{N}$$

records *one chosen branch* and its number. Both fibers happen to be $\mathbb{N}$, yet choosing Π or Σ makes a decisive difference. The propositions-as-types interpretation similarly distinguishes a dependent sum $\sum_{x:A} P(x)$, containing a witness and evidence for $P(x)$, from its *propositional truncation*, which asserts mere existence without exposing a particular witness. This distinction is essential if one hopes to extract an algorithm from a proof.

§1.11 Univalence preserves the symmetries of an identity

The Boolean type has precisely two self-equivalences:

$$\mathrm{id} : \mathsf{Bool} \simeq \mathsf{Bool}, \qquad \mathrm{swap} : \mathsf{Bool} \simeq \mathsf{Bool}.$$

Intensional type theory represents an identification of two types A, B in a universe $\mathcal{U}$ by the identity type $(A =_{\mathcal{U}} B)$. Every such identification induces an equivalence of types:

$$(A =_{\mathcal{U}} B) \longrightarrow \mathsf{Equiv}(A, B).$$

The univalence axiom says that this canonical map is an equivalence. In a univalent universe,

$$\boxed{(A =_{\mathcal{U}} B) \simeq \mathsf{Equiv}(A, B).}$$

Setting $A = B = \mathsf{Bool}$ makes the meaning surprisingly tangible:

$$(\mathsf{Bool} =_{\mathcal{U}} \mathsf{Bool}) \simeq \mathsf{Equiv}(\mathsf{Bool}, \mathsf{Bool}).$$

There are two distinguishable elements on the right, identity and swap; hence the *universe-level identity type* of Bool with itself has two distinct paths under univalence.

The nontrivial automorphism becomes a way of identifying the type with itself; it does not disappear into one anonymous equality.[2]

The qualification matters twice. This is an identity *of types in a universe*, not a claim that $\mathsf{true} = \mathsf{false}$ inside Bool. And an arbitrary bijection between underlying sets need not be an isomorphism of groups, topological spaces, or ordered sets: preserving the extra structure is part of the type of equivalence one is discussing.

The same little two-point calculation has now exhibited three ways to ask a precise question. Set theory distinguishes membership and equality of sets; category theory distinguishes maps and universal characterizations; dependent type theory turns the existence of data into an explicit construction. Univalence then makes equivalences visible as possible identifications, including their nontrivial symmetries.

[2] See the Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, Section 2.10.